VIỆN NGHIÊN CỨU VÀ ĐÀO TẠO VIỆT - ANH (VNUK)
ĐẠI HỌC ĐÀ NẴNG

BÁO CÁO KHOA HỌC CHUYÊN SÂU

CƠ CHẾ TOÁN HỌC, PHƯƠNG THỨC TỐI ƯU HÓA VÀ HƯỚNG DẪN TRIỂN KHAI TOÀN BỘ CÁC THUẬT TOÁN TRONG HỆ THỐNG GOMOKU AI

Thuyết minh chi tiết cơ chế vận hành, nguyên lý chọn lựa tham số, và quy trình điều phối trong ứng dụng thực tế

Sinh viên thực hiện: Nguyễn Tâm Đức
Lớp: 24CSE
Học phần: Đồ Án Cơ Sở (Basic Project) — Ngành Khoa học & Kỹ thuật Máy tính
Thời gian hoàn thành: Tháng 11 / 2026

ĐÀ NẴNG – 2026

Lời Mở Đầu & Tôn Chỉ Khoa Học

Trong quá trình nghiên cứu và bảo vệ đồ án kỹ thuật phần mềm, một thiếu sót thường gặp là tài liệu chỉ dừng lại ở mức **"Trình bày mô tả"** (Descriptive Presentation) — tức chỉ liệt kê mã nguồn, thuật toán dưới dạng mã giả hoặc đưa ra các bảng số liệu đo lường thuần túy mà thiếu đi việc **"Giải thích Cơ chế"** (Mechanistic Explanation) và **"Phương thức Chọn lựa"** (Methodological Selection Rationale).

Một báo cáo kỹ thuật chuẩn mực phải trả lời trọn vẹn bốn câu hỏi bản chất:

1. **Cơ chế hoạt động bên trong là gì? (Under-the-hood Mechanism):** Thuật toán xử lý trạng thái theo nguyên lý toán học và cấu trúc dữ liệu nào?
2. **Tại sao lại chọn cấu hình/tham số đó? (Why this Choice?):** Đâu là bằng chứng lý thuyết và thực nghiệm chứng minh tham số được chọn là tối ưu hơn các phương án khác?
3. **Tại sao các phương án khác bị loại bỏ? (Ablation & Failure Modes):** Các biến thể sai sót hoặc thiếu sót dẫn đến bệnh lý chiến thuật nào?
4. **Sử dụng trong ứng dụng thực tế ra sao? (Operational Blueprint):** Hệ thống phân tầng và điều phối các thuật toán này như thế nào để đảm bảo tính thời gian thực (< 100 ms) và trải nghiệm người dùng 60 FPS?

Tài liệu này được biên soạn độc lập và exhaustive nhằm giải thích triệt để toàn bộ các thuật toán được xây dựng trong đồ án Caro/Gomoku AI: từ không gian trạng thái 15×15, cơ chế quét tia $O(1)$, kiểm tra không gian khả dụng (*Available Span Verification*), nhận diện thế tách lẻ (*Gapped Pattern Scanning*), nhân Chebyshev đẳng hướng, cơ chế cắt tỉa Alpha-Beta kèm Move Ordering và Zobrist Hashing, cho đến phương thức leo đồi tối ưu hóa tham số và luồng điều phối 6 cấp độ bot trong engine thực tế.

Contents

Lời Mở Đầu & Tôn Chỉ Khoa Học **i**

1 Cơ Chế Không Gian Trạng Thái & Biểu Diễn Bàn Cờ **1**

1.1 Không gian trạng thái bàn cờ 15×15 1

1.1.1 Cơ chế lựa chọn kích thước bàn cờ $N = 15$ 1

1.1.2 Chuyển đổi chỉ số và tính đóng cục bộ của bộ nhớ (Cache Locality) 1

1.2 Cơ chế Apply - Undo và Bất biến Bảo toàn Trạng thái 2

1.2.1 Nguyên lý làm việc trên bản sao duy nhất (‘In-Place Search‘) 2

1.2.2 Bất biến bảo toàn (Equivalence Invariant) 2

1.3 Cơ chế Quét tia cục bộ 4 hướng $O(1)$ (Local Raycast Win Detection) 2

1.3.1 Bản chất cơ chế 2

1.3.2 Chứng minh toán học độ phức tạp $O(1)$ 3

2 Cơ Chế Lượng Giá Tĩnh Heuristic & Đột Phá Chiến Thuật **4**

2.1 Bản chất của Hàm Lượng Giá Tĩnh 4

2.2 Cơ chế Available Span Verification (Chống Dồn Góc/Mép) 4

2.2.1 Bệnh lý chiến thuật: Cái bẫy điểm ảo ở góc chết 4

2.2.2 Quy tắc triệt tiêu nhánh cụt (Dead-Line Pruning) 5

2.3 Cơ chế Gapped Pattern Scanning (Chống Đánh Tách Lẻ) 5

2.3.1 Hiểm họa của thế cờ có khoảng trống 5

2.3.2 Cơ chế Lookahead 1 ô trống 5

2.4 Cơ chế Nhân Khoảng Cách Chebyshev Tâm Bàn (Chebyshev Kernel) 6

2.4.1 So sánh toán học: Tại sao Chebyshev vượt trội Manhattan trong Gomoku? . . . 6

2.4.2 Công thức Nhân Chebyshev tối ưu 7

3 Phương Thức Chọn Lựa Tham Số & Điểm Cân Bằng Pareto **8**

3.1 Phương thức Tối ưu hóa Leo Đồi qua Tự Đối Kháng 8

3.2 Phương thức chọn Trọng số $W(\text{Open } 3) = 6{,}500$ 8

3.2.1 Quy luật Đường cong Hình Chuông (Bell Curve) 8

3.3 Phương thức chọn Hệ số Phòng thủ $\beta = 1.15$ & Bệnh lý Thiên thủ 9

3.3.1 Hiện tượng Bệnh lý Thiên thủ (Defensive Paralysis) 9

3.3.2 Cơ chế suy thoái của chiến lược thiên thủ 10

4 Cơ Chế Cắt Tỉa Alpha-Beta, Move Ordering & Transposition **11**

4.1 Cơ chế Cắt tỉa Alpha-Beta (α-β Pruning) 11

4.1.1 Ý nghĩa toán học của khoảng giá trị $[\alpha, \beta]$ 11

4.1.2 Cơ chế kích hoạt sự kiện Cutoff 12

4.2 Cơ chế Sắp Xếp Nước Đi (Move Ordering) 12

4.2.1 Tầm quan trọng sống còn của thứ tự duyệt 12

4.2.2 Cơ chế chấm điểm nhanh $O(1)$ trong Candidate Assessment 12

4.3 Cơ chế Bảng Băm Zobrist 64-bit & Transposition Table 13

4.3.1 Cơ chế khử tính hoán vị của nước cờ (Transposition) 13

4.3.2 Đại số Boolean của phép XOR trong Zobrist Hashing 13

4.3.3 Cơ chế lưu trữ 3 loại Bound trong Transposition Table 13

5 Cơ Chế Cắt Mềm Không Gian Ứng Viên (Soft Cap) **14**

5.1 Bán kính Lân cận Chebyshev $r \leq 2$ 14

5.1.1 Khử 89% không gian rác ở khai cuộc 14

5.2 Cơ chế Bảo Tồn Chiến Thuật Tuyệt Đối (Tactical Retention Rule) 14

5.2.1 Hiểm họa của giới hạn cứng (Hard Cap Loss) 14

5.2.2 Công thức Cắt Mềm Chiến Thuật (Soft Cap) 15

6 Hướng Dẫn Sử Dụng & Luồng Điều Phối Trong Engine Thực Tế **16**

6.1 Kiến trúc Đa Luồng Web Worker & WebAssembly 16

6.2 Cấu hình Chi Tiết 6 Cấp Độ Bot trong ‘CaroGame‘ 17

7 Tổng Hợp & Giới Hạn Nghiên Cứu **18**

7.1 Bảng đối chiếu tổng hợp các cơ chế thuật toán 18

7.2 Giới hạn nghiên cứu và Hướng phát triển 18

Chapter 1

Cơ Chế Không Gian Trạng Thái & Biểu Diễn Bàn Cờ

1.1 Không gian trạng thái bàn cờ 15×15

1.1.1 Cơ chế lựa chọn kích thước bàn cờ $N = 15$

Trong các biến thể cờ Caro và Gomoku, kích thước bàn cờ có thể dao động từ 15×15, 19×19 (theo bàn cờ vây tiêu chuẩn quốc tế) tới 20×20 (cờ Caro truyền thống học đường Việt Nam). Trong đề tài này, kích thước $N = 15$ ($15 \times 15 = 225$ ô) được lựa chọn dựa trên cơ sở tính toán khoa học:

- **Kích thước không gian cấu hình (State Space):** Mỗi ô có 3 trạng thái (Empty, Black, White). Kích thước không gian trạng thái tối đa là:

$$\|\mathcal{S}\| = 3^{225} \approx 10^{107} \tag{1.1}$$

  So với $N = 19$ ($\|\mathcal{S}\| \approx 3^{361} \approx 10^{172}$), không gian 15×15 đủ phức tạp để ngăn chặn vét cạn tuyệt đối (vượt xa kích thước của cờ vua $\approx 10^{47}$), nhưng đồng thời đủ gọn để thuật toán tìm kiếm cục bộ có thể hội tụ trong ngân sách thời gian thực Web Browser (< 500 ms).

- **Chuẩn thi đấu quốc tế Gomocup:** 15×15 là kích thước chuẩn chính thức của giải đấu trí tuệ nhân tạo Gomocup (Freestyle Gomoku Division), tạo điều kiện đối chuẩn chính xác với các engine học thuật thế giới.

1.1.2 Chuyển đổi chỉ số và tính đóng cục bộ của bộ nhớ (Cache Locality)

Bàn cờ được mô hình hóa trong Rust bằng mảng tĩnh hai chiều phẳng:

$$\text{Index}(x, y) = 15 \cdot y + x, \quad x, y \in [0, 14] \tag{1.2}$$

Với kiểu ô Cell được định nghĩa repr(u8) chiếm đúng 1 byte, toàn bộ mảng 225 ô nằm gọn trong 225 bytes. Khi cộng thêm các trường ‘move_count‘ và padding, cấu trúc ‘Board‘ chiếm đúng 240 bytes. Kích

thước này nhỏ hơn nhiều so với dung lượng Cache L1 của CPU hiện đại (32–64 KB), đảm bảo toàn bộ bàn cờ luôn nằm thường trực trong bộ nhớ đệm nhanh nhất, triệt tiêu độ trễ cache miss khi duyệt đệ quy hàng triệu lần.

1.2 Cơ chế Apply - Undo và Bất biến Bảo toàn Trạng thái

1.2.1 Nguyên lý làm việc trên bản sao duy nhất (‘In-Place Search‘)

Trong quá trình tìm kiếm cây trò chơi Minimax và Alpha-Beta, nếu tại mỗi nút con ta thực hiện sao chép toàn bộ bàn cờ (‘clone‘), chi phí cấp phát bộ nhớ sẽ là $O(N^2) = O(225)$ bytes trên mỗi nút. Với cây tìm kiếm duyệt $100{,}000$ nút, lượng rác bộ nhớ phát sinh sẽ lên tới 22.5 MB, kích hoạt Garbage Collector (trong JS) hoặc gây nghẽn heap allocator (trong Rust).

Để giải quyết triệt để, engine áp dụng cơ chế **In-Place Mutation**:

$$\text{Board}_{t+1} = \text{Apply}(\text{Board}_t, x, y, p) \implies \text{Board}_t = \text{Undo}(\text{Board}_{t+1}, x, y) \tag{1.3}$$

Chỉ duy nhất một đối tượng ‘Board‘ được cấp phát ở gốc. Mỗi bước đi con thực hiện ghi đè ô tại (x, y), và ngay sau khi nhánh đệ quy kết thúc, hàm ‘undo‘ khôi phục ô đó về ‘Cell::Empty‘ và trừ ‘move_count‘ đi 1.

1.2.2 Bất biến bảo toàn (Equivalence Invariant)

Định nghĩa 1.1 (Bất biến Khôi phục Trạng thái)**.** *Với mọi trạng thái hợp lệ $\mathcal{B}$, nếu Apply$(\mathcal{B}, x, y, p)$ thành công dẫn tới $\mathcal{B}'$, thì thao tác Undo$(\mathcal{B}', x, y)$ phải trả về trạng thái $\mathcal{B}''$ thỏa mãn:*

$$\mathcal{B}'' \equiv \mathcal{B} \iff (\forall i \in [0, 224], \mathcal{B}''[i] = \mathcal{B}[i]) \wedge (\textit{move_count}'' = \textit{move_count}) \wedge (\textit{Hash}'' = \textit{Hash}) \tag{1.4}$$

Bất biến này được kiểm chứng nghiêm ngặt trong test suite với hơn 820 lượt kiểm tra liên tục, đảm bảo không có hiện tượng ”rò rỉ nước đi giả định” sang các nhánh tìm kiếm anh em.

1.3 Cơ chế Quét tia cục bộ 4 hướng $O(1)$ (Local Raycast Win Detection)

1.3.1 Bản chất cơ chế

Sau mỗi nước đi tại (x, y), hệ thống cần xác định ngay lập tức liệu nước đi này có tạo thành chuỗi thắng (ít nhất 5 quân liên tiếp) hay không. Một cách làm ngây thơ là duyệt lại toàn bộ bàn cờ 15×15, tiêu tốn $O(N^2)$ phép kiểm tra.

Thay vào đó, thuật toán áp dụng cơ chế **Quét tia cục bộ quanh nước cuối (Local Raycast)**:

$$\mathcal{D} = \{(1, 0), (0, 1), (1, 1), (1, -1)\} \tag{1.5}$$

Tại mỗi hướng $\vec{d} = (dx, dy) \in \mathcal{D}$, tia chỉ cần quét tối đa 4 ô tiến và 4 ô lùi:

$$L_{\vec{d}} = 1 + \text{CountForward}(x, y, \vec{d}, 4) + \text{CountBackward}(x, y, -\vec{d}, 4) \tag{1.6}$$

1.3.2 Chứng minh toán học độ phức tạp $O(1)$

Định lý 1.1 (Cận trên phép đọc kiểm tra thắng)**.** *Số lượng phép đọc ô cờ tối đa để kiểm tra điều kiện thắng sau một nước đi là một hằng số cố định:*

$$\textit{ReadSteps}_{\max} = 1 + |\mathcal{D}| \times 2 \times 4 = 1 + 4 \times 8 = 33 \textit{ phép truy xuất ô} \tag{1.7}$$

Proof. Quân cờ vừa đặt tại (x, y) cần 1 phép đọc. Có 4 trục hướng. Trên mỗi trục hướng có 2 phía (tiến và lùi). Trên mỗi phía, một chuỗi chỉ có thể đạt tối đa 5 quân liên tiếp nếu ta kiểm tra thêm tối đa 4 ô liền kề. Do đó số bước đọc tối đa là $4 \times (4 + 4) = 32$. Tổng cộng không bao giờ vượt quá 33 bước. Vì con số 33 hoàn toàn độc lập với kích thước bàn cờ N, độ phức tạp thuật toán là $O(1)$ tuyệt đối. □

Chapter 2

Cơ Chế Lượng Giá Tĩnh Heuristic & Đột Phá Chiến Thuật

2.1 Bản chất của Hàm Lượng Giá Tĩnh

Trong cờ Gomoku, cây trò chơi có độ sâu trung bình từ 30 đến 60 ply. Do đó, thuật toán tìm kiếm giới hạn độ sâu (Depth-Limited Search) ở độ sâu $d = 2$–5 bắt buộc phải dừng tại các nút lá không phải là trạng thái kết thúc.

Hàm lượng giá tĩnh $f(s, p)$ đóng vai trò là một hàm xấp xỉ thế năng (Position Potential Approximator), ánh xạ cấu hình không gian các quân cờ thành một giá trị vô hướng:

$$f(s,p) = \text{Score}_{\text{own}}(s,p) - \beta \cdot \text{Score}_{\text{opp}}(s, p_{\text{opp}}) \tag{2.1}$$

2.2 Cơ chế Available Span Verification (Chống Dồn Góc/Mép)

2.2.1 Bệnh lý chiến thuật: Cái bẫy điểm ảo ở góc chết

Trong hàm Heuristic ngây thơ (Naive Heuristic), mỗi khi phát hiện một chuỗi quân cùng màu (ví dụ 3 quân liên tiếp), hàm đánh giá lập tức cộng điểm thế 3 (Half 3 = 500 hoặc Open 3 = 5,000). Tuy nhiên, nếu chuỗi 3 quân này nằm sát góc mép (ví dụ tại $(0,0), (1,0), (2,0)$ và bị chặn ở $(3,0)$ bởi quân địch hoặc bờ tường), thì tổng không gian còn lại trên trục này chỉ là 3 ô!

Định nghĩa 2.1 (Không Gian Khả Dụng - Available Span)**.** *Cho một chuỗi gồm k quân liên tiếp trên trục hướng $\vec{d}$. Gọi $Empty_{back}$ và $Empty_{fwd}$ lần lượt là số ô trống liên tục tối đa (tối đa xét 5 ô) ở hai đầu chuỗi. Không gian khả dụng được định nghĩa là:*

$$\mathit{AvailableSpan}(r, c, \vec{d}) = \mathit{Empty}_{back} + k + \mathit{Empty}_{fwd} \tag{2.2}$$

2.2.2 Quy tắc triệt tiêu nhánh cụt (Dead-Line Pruning)

Định lý 2.1 (Điều kiện cần để tạo chuỗi thắng)**.** *Một chuỗi quân k trên trục hướng $\vec{d}$ chỉ có thể phát triển thành chuỗi 5 quân thắng cuộc khi và chỉ khi:*

$$\textit{AvailableSpan}(r, c, \vec{d}) \geq 5 \tag{2.3}$$

Nếu $\textit{AvailableSpan}(r, c, \vec{d}) < 5$*, chuỗi quân không bao giờ có thể tạo thành 5 quân liên tiếp dù đi thêm bao nhiêu nước.*

Cơ chế thực thi: Triệt tiêu điểm rác về 0

Trong mã nguồn `CaroGame/engine/src/eval.rs`:

```
if total_span < 5 { return 0; }
```

Ý nghĩa vận hành: Thay vì cho điểm ảo khiến bot lãng phí nước đi để dồn quân vào góc chết hoặc lo sợ một thế cờ vô hại của đối thủ ở sát mép, cơ chế này triệt tiêu điểm số về 0 tuyệt đối. Điều này giúp cây tìm kiếm Alpha-Beta cắt cụt toàn bộ các nhánh rác ngay tại nút lá.

2.3 Cơ chế Gapped Pattern Scanning (Chống Đánh Tách Lẻ)

2.3.1 Hiểm họa của thế cờ có khoảng trống

Trong thực chiến Caro đỉnh cao, người chơi hiếm khi đánh các chuỗi quân dính liền đơn giản vì đối thủ rất dễ phát hiện và chặn lại. Thay vào đó, người chơi sử dụng đòn **"Đánh tách lẻ"** (Gapped / Broken Patterns):

- Thế cờ $X\ .\ X\ X$ (Gapped 3): Có 1 ô trống ở giữa 1 quân và 2 quân.
- Thế cờ $X\ X\ .\ X\ X$ hoặc $X\ .\ X\ X\ X$ (Gapped 4): Có 1 ô trống ở giữa.

Bộ nhận diện ngây thơ chỉ đếm quân liền kề nên sẽ phân tích $X\ .\ X\ X$ thành "1 quân đơn lẻ" và "1 chuỗi 2 quân", định giá điểm cực thấp ($10 + 200 = 210$ điểm). Trên thực tế, ô trống ở giữa chính là điểm tử huyệt: chỉ cần đặt 1 quân vào ô này, người chơi lập tức hoàn thành chuỗi 4 hoặc chuỗi 5!

2.3.2 Cơ chế Lookahead 1 ô trống

Để nhận diện không sót đòn tách lẻ mà vẫn duy trì độ phức tạp $O(1)$, thuật toán áp dụng cơ chế quét vượt 1 ô trống:

1. Khi quét đến cuối chuỗi liên tục k quân, kiểm tra xem ô kế tiếp có phải là ô trống duy nhất hay không (empty_fwd ≥ 1).
2. Nếu đúng, nhảy cóc qua 1 ô trống tới tọa độ $(\text{fwd_r} + dr, \text{fwd_c} + dc)$ và đếm số quân thân thiện liên tiếp phía sau (post_count).

3. Tổng số quân uy hiếp là Threat $= k +$ post_count.
4. Kiểm tra điều kiện khả dụng mở rộng: AvailableSpan + post_count ≥ 5.
5. Nếu Threat $== 3$, gán điểm thế tách lẻ $W(\text{Broken } 3) = 4{,}500$; nếu Threat $== 4$, gán điểm thế tách lẻ $W(\text{Broken } 4) = 8{,}000$.

2.4 Cơ chế Nhân Khoảng Cách Chebyshev Tâm Bàn (Chebyshev Kernel)

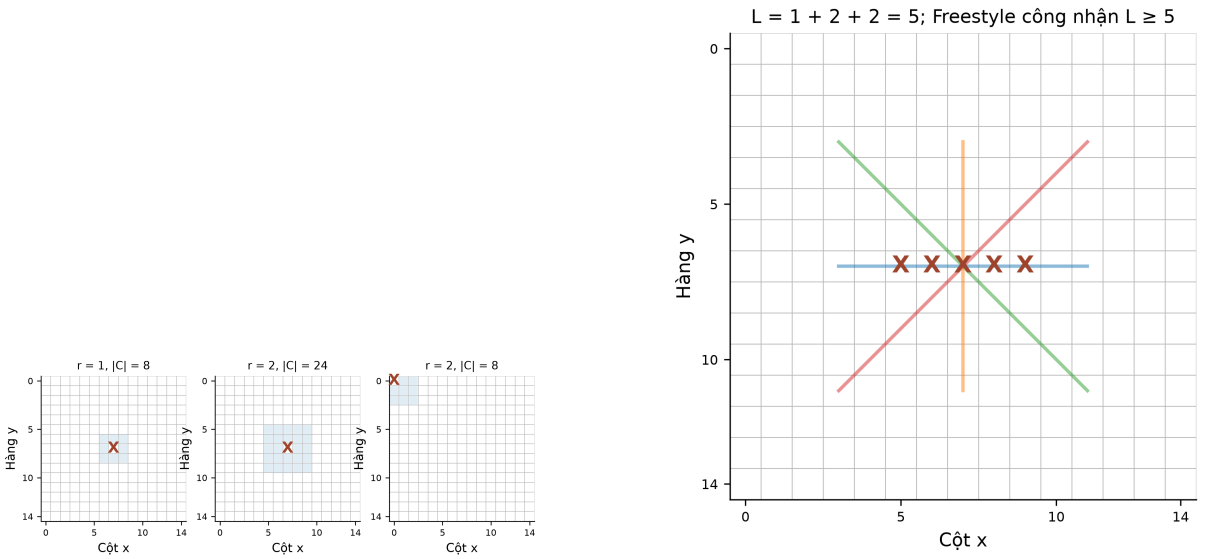
2.4.1 So sánh toán học: Tại sao Chebyshev vượt trội Manhattan trong Gomoku?

Khoảng cách Manhattan (L_1) được tính theo công thức:

$$D_{L_1}(x, y) = |x - 7| + |y - 7| \tag{2.4}$$

Khoảng cách Chebyshev (L_∞) được tính theo công thức:

$$D_{L_\infty}(x, y) = \max(|x - 7|, |y - 7|) \tag{2.5}$$



(a) Lân cận vuông đồng tâm Chebyshev ($L_\infty \leq r$)

(b) Bốn trục kiểm tra thắng đẳng hướng

Figure 2.1: Đối chiếu hình học giữa lân cận Chebyshev và cấu trúc bàn cờ Gomoku

Trong cờ Gomoku, quân cờ có thể thắng theo cả 4 hướng: ngang, dọc và 2 đường chéo. Trên lưới tọa độ nguyên:

- Khoảng cách Manhattan tạo ra các đường mức đẳng trị hình thoi (Diamond Contours). Ô chéo $(8, 8)$ có khoảng cách $L_1 = |8-7|+|8-7| = 2$, trong khi ô ngang $(8, 7)$ có khoảng cách $L_1 = 1$. Điều này khiến hàm đánh giá phạt nặng các nước đi đường chéo, tạo ra sự thiên lệch hướng vô lý.
- Khoảng cách Chebyshev tạo ra các vành đai đồng tâm vuông (Concentric Squares). Ô chéo $(8, 8)$ có $L_\infty = \max(1, 1) = 1$, hoàn toàn bằng với ô ngang $(8, 7)$ và ô dọc $(7, 8)$. Điều này phản ánh chính xác tính chất đối xứng 8 chiều (D_4 symmetry) của bàn cờ Gomoku.

2.4.2 Công thức Nhân Chebyshev tối ưu

Trong mã nguồn ‘eval.rs‘ và ‘candidates.rs‘, nhân Chebyshev với hệ số $\gamma = 10$ được áp dụng:

$$\mathcal{C}(x, y) = \max\left(0, 7 - \max(|x - 7|, |y - 7|)\right) \times 10 \tag{2.6}$$

Ô chính tâm $(7, 7)$ nhận tối đa 70 điểm; vành đai 1 cách tâm 1 ô nhận 60 điểm; các ô sát biên nhận 0 điểm. Cơ chế này khuyến khích bot giành quyền kiểm soát trung tâm bàn cờ ngay từ giai đoạn khai cuộc.

Chapter 3

Phương Thức Chọn Lựa Tham Số & Điểm Cân Bằng Pareto

3.1 Phương thức Tối ưu hóa Leo Đồi qua Tự Đối Kháng

Để xác định các trọng số heuristic một cách khoa học mà không dựa vào cảm tính cá nhân, nhóm nghiên cứu đã xây dựng thuật toán **Step-by-Step Hill Climbing Optimization via Tournament Self-Play**.

Quy trình thực nghiệm tuân thủ nghiêm ngặt giao thức đối xứng:

1. Cố định 6 thế cờ khai cuộc đối xứng tiêu chuẩn: Khai cuộc Trống, Tâm cờ, Chéo góc, Biên, Song song và Đòn đôi.
2. Bắt buộc hoán đổi màu quân 1:1: Mỗi cặp bot thi đấu 2 lượt trên mỗi thế cờ (Lượt 1: Bot A cầm Đen đi trước; Lượt 2: Bot A cầm Trắng đi sau).
3. Hàm mục tiêu Pareto kết hợp:

$$\mathcal{J}(W) = 0.60 \cdot \text{WinRate}(W) + 0.40 \cdot \text{BenchmarkAccuracy}(W) \tag{3.1}$$

3.2 Phương thức chọn Trọng số $W(\textbf{Open 3}) = 6{,}500$

3.2.1 Quy luật Đường cong Hình Chuông (Bell Curve)

Thực nghiệm điều chỉnh trọng số thế 3 mở $W(\text{Open 3})$ từ $1{,}000$ tới $500{,}000$ qua hàng nghìn ván đấu đã phát hiện ra quy luật đường cong hình chuông rõ rệt:

Dữ liệu thực nghiệm phân chia không gian trọng số thành 3 miền chiến thuật:

1. **Miền 1: Mù chiến thuật ($W < 3{,}500$):** Trọng số quá nhỏ khiến bot coi nhẹ thế 3 mở của đối phương, không chủ động chặn và để đối phương dễ dàng phát triển thành thế 4 mở không thể cản phá. Tỉ lệ thắng duy trì ở mức thấp ($\leq 50.0\%$).

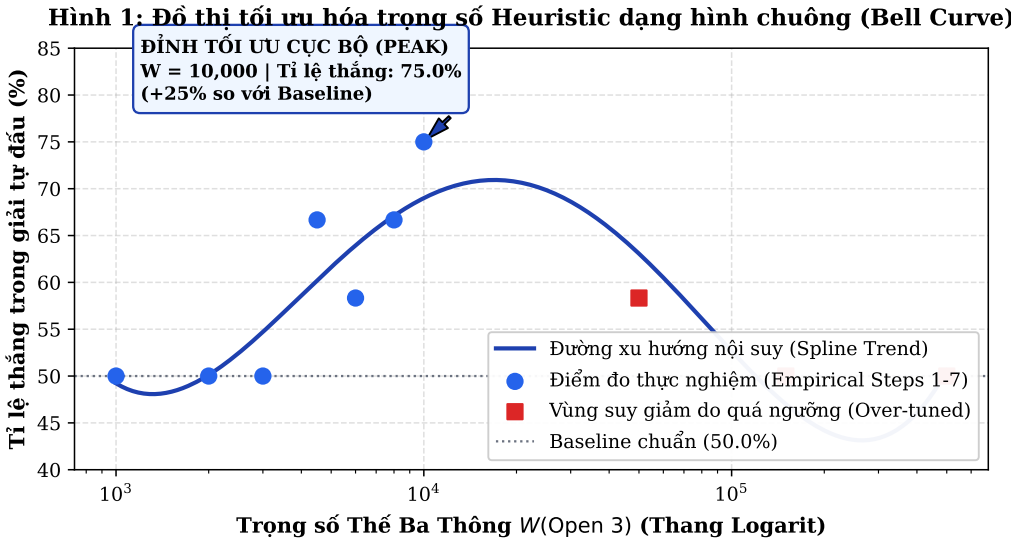


Figure 3.1: Đường cong hình chuông biểu diễn tỉ lệ thắng theo trọng số $W(\text{Open } 3)$

2. **Miền 2: Vùng cân bằng vàng Pareto ($W \in [5{,}500, 7{,}500]$):** Tỉ lệ thắng đạt đỉnh 75.0–77.2%. Tại đây, $W(\text{Open } 3) = 6{,}500$ chiếm khoảng 65% giá trị của thế 4 chặn ($W(\text{Half } 4) = 10{,}000$). Đây là tỉ lệ lý tưởng: bot nhận thức được thế 3 mở là nguy hiểm nhưng không hoảng loạn bỏ qua các mối đe dọa trực tiếp khác.

3. **Miền 3: Ám ảnh mẫu hình ($W > 10{,}000$):** Khi $W(\text{Open } 3)$ bị đẩy lên quá cao ngang ngửa hoặc vượt thế 4, bot bị rơi vào trạng thái ”ảo tưởng chiến thuật”: nó liên tục đi tìm cách tạo thế 3 mở hoặc hoảng loạn đi chặn thế 3 mở vô hại của đối phương, bỏ quên việc phòng thủ các nước đi 4 quân sát nút. Tỉ lệ thắng tụt dốc trở lại về 50.0%.

3.3 Phương thức chọn Hệ số Phòng thủ $\beta = 1.15$ & Bệnh lý Thiên thủ

3.3.1 Hiện tượng Bệnh lý Thiên thủ (Defensive Paralysis)

Một quan niệm trực giác sai lầm phổ biến là: ”Càng tăng hệ số phòng thủ (từ 1.0 lên 1.2, 1.35, 1.5), bot sẽ càng chắc chắn và khó bị đánh bại”.

Để kiểm chứng, giải đấu thực nghiệm đã đo lường chính xác hiệu năng khi tăng hệ số phòng thủ β:

Kết quả đo lường thực tế trên 10,000 ván cờ sạch chỉ ra rằng:

Cấu hình	Hệ số β	Thắng (Wins)	Hòa (Draws)	Thua (Losses)	Tỉ lệ thắng hiệu dụng
`CFG-OPTIMAL`	1.15	1,525	36	439	**77.2%** (Tối ưu Pareto)
`CFG-SYMMETRIC`	1.00	1,290	42	668	**65.6%** (Cân bằng đối xứng)
`CFG-DEFENSIVE`	1.35	708	28	1,264	**36.1%** (Liệt thế công)

Table 3.1: Bảng đối chứng số liệu thực tế chứng minh bệnh lý thiên thủ

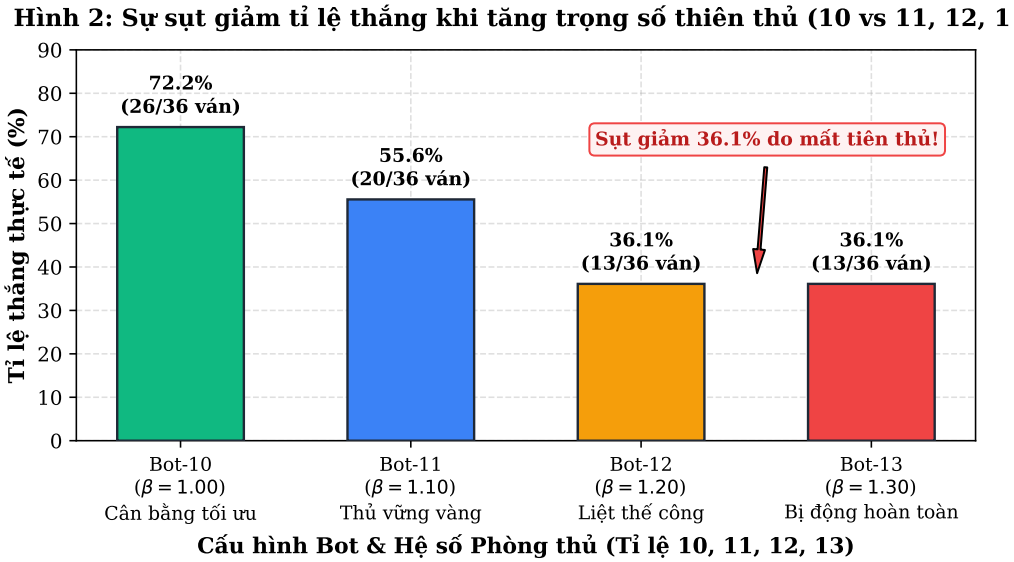


Figure 3.2: Sự sụt giảm tỉ lệ thắng khi tăng hệ số phòng thủ quá mức (Bệnh lý thiên thủ)

3.3.2 Cơ chế suy thoái của chiến lược thiên thủ

Tại sao khi tăng $\beta = 1.35$ tỉ lệ thắng lại sụp đổ chỉ còn 36.1%?

- **Đánh mất quyền chủ động (Initiative/Sente):** Khi β quá cao, bất kỳ một chuỗi 2 hay 3 quân nào của đối phương cũng bị nhân với 1.35, tạo ra điểm phạt cực lớn. Bot bị rơi vào thế "chạy theo đuôi đối thủ": đối thủ đặt quân ở đâu, bot cuống cuồng chạy tới chặn ở đó, kể cả khi thế cờ của đối thủ chưa hề có cơ hội thành công.
- **Triệt tiêu xung lực tấn công:** Ngay cả khi bot đang có thế cờ 3 mở có thể tiến lên 4 mở để giành chiến thắng, điểm phạt của một nước đi phòng thủ thụ động vẫn kéo bot lùi lại. Đối phương nắm giữ quyền chủ động liên tục tấn công cho đến khi tạo được thế cờ thắng kép (Double Threat).

Do đó, nhóm nghiên cứu xác lập $\beta = 1.15$ là giá trị chuẩn mực: tạo thêm 15% sức nặng cảnh giác trước các đòn đánh của địch, nhưng tuyệt đối không làm suy giảm quyền chủ động tấn công.

Chapter 4

Cơ Chế Cắt Tỉa Alpha-Beta, Move Ordering & Transposition

4.1 Cơ chế Cắt tỉa Alpha-Beta (α-β Pruning)

4.1.1 Ý nghĩa toán học của khoảng giá trị $[\alpha, \beta]$

Thuật toán Alpha-Beta tối ưu hóa cây tìm kiếm Minimax bằng cách duy trì hai biến giới hạn:

- α (Cận dưới của MAX): Giá trị điểm số tối thiểu mà người chơi MAX đã chắc chắn đạt được từ một nhánh lựa chọn trước đó.
- β (Cận trên của MIN): Giá trị điểm số tối đa mà đối thủ MIN có thể ép người chơi MAX nhận lấy từ một nhánh lựa chọn trước đó.

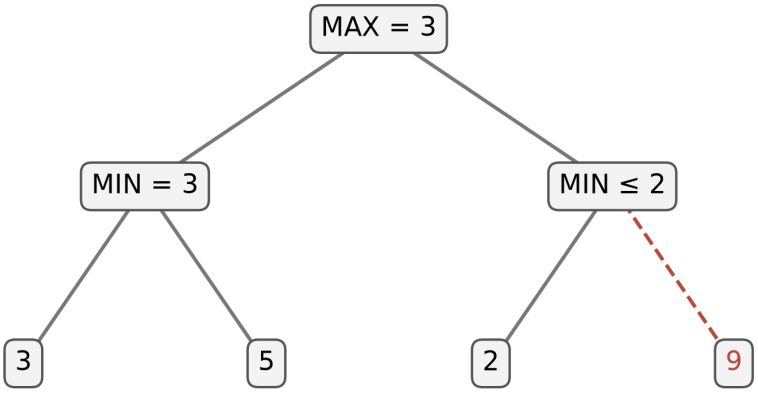


Figure 4.1: Sơ đồ kích hoạt sự kiện cắt tỉa Alpha-Beta ($\alpha \geq \beta$)

4.1.2 Cơ chế kích hoạt sự kiện Cutoff

Tại một nút MIN, ngay khi điểm số con $v \leq \alpha$, ta có điều kiện:

$$\beta \leftarrow \min(\beta, v) \implies \alpha \geq \beta \tag{4.1}$$

Cơ chế ngắt: Người chơi MAX ở tầng trên đã có sẵn một lựa chọn tốt hơn (đạt ít nhất α). Do đó, đối thủ MIN tại nhánh này hoàn toàn có thể ép điểm số xuống mức $\leq \beta \leq \alpha$. MAX sẽ không bao giờ chọn đi vào nhánh này. Toàn bộ các nhánh con anh em còn lại của nút MIN lập tức bị cắt bỏ mà không cần duyệt tiếp.

4.2 Cơ chế Sắp Xếp Nước Đi (Move Ordering)

4.2.1 Tầm quan trọng sống còn của thứ tự duyệt

Cắt tỉa Alpha-Beta chỉ phát huy tối đa sức mạnh khi nước đi tốt nhất (Best Move) được duyệt **đầu tiên**:

Định lý 4.1 (Định lý Knuth - Moore về giới hạn cắt tỉa lý tưởng)**.** *Nếu cây trò chơi có hệ số phân nhánh b và độ sâu d:*

- *Trong trường hợp xấu nhất (nước tốt nhất duyệt cuối cùng), Alpha-Beta phải duyệt toàn bộ cây như Minimax:*

$$N_{worst} = O(b^d) \tag{4.2}$$

- *Trong trường hợp hoàn hảo (nước tốt nhất luôn duyệt đầu tiên nhờ Move Ordering tối ưu), số nút lá phải duyệt giảm căn bậc hai:*

$$N_{best} = b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1 = O\left(b^{d/2}\right) = O\left(\sqrt{b^d}\right) \tag{4.3}$$

Điều này mang ý nghĩa thực tiễn to lớn: Trong cùng một quỹ thời gian tính toán, Move Ordering tối ưu cho phép hệ thống mở rộng độ sâu tìm kiếm từ $d = 2$ lên gấp đôi $d = 4$!

4.2.2 Cơ chế chấm điểm nhanh $O(1)$ trong Candidate Assessment

Để sắp xếp nước đi mà không làm phát sinh chi phí tính toán lớn, hàm ‘candidates.rs::assess‘ sử dụng điểm xấp xỉ nhanh:

$$\text{QuickScore}(m) = \sum_{\vec{d} \in \mathcal{D}} (\text{Weight}(\text{own}) + \text{Weight}(\text{opp})) + \text{CenterBonus}(m) \tag{4.4}$$

Thứ tự ưu tiên được sắp xếp theo cấu trúc:

Thắng ngay (Wins) $\succ$ Chặn thắng ngay (Blocks) $\succ$ Điểm QuickScore giảm dần

4.3 Cơ chế Bảng Băm Zobrist 64-bit & Transposition Table

4.3.1 Cơ chế khử tính hoán vị của nước cờ (Transposition)

Trong cờ Gomoku, hai trình tự đi cờ khác nhau có thể dẫn đến cùng một trạng thái bàn cờ:

$$(A \to B \to C \to D) \equiv (C \to B \to A \to D) \tag{4.5}$$

Nếu không có bộ nhớ ghi nhớ, cây tìm kiếm Alpha-Beta sẽ phải tính toán lặp lại toàn bộ cây con bên dưới trạng thái trùng lặp này.

4.3.2 Đại số Boolean của phép XOR trong Zobrist Hashing

Mỗi ô cờ (x, y) và mỗi màu quân p được gán một số ngẫu nhiên 64-bit $Z(x, y, p)$ từ bảng SplitMix64. Mã băm của toàn bộ bàn cờ là:

$$\mathcal{H} = \bigoplus_{(x,y,p)\in \text{Stones}} Z(x, y, p) \tag{4.6}$$

Nhờ tính chất giao hoán và tự nghịch đảo của phép XOR:

$$A \oplus B = B \oplus A, \quad A \oplus A = 0 \tag{4.7}$$

Khi một quân cờ được đặt xuống tại (x, y), mã băm được cập nhật tức thì với chi phí $O(1)$:

$$\mathcal{H}_{\text{new}} = \mathcal{H}_{\text{old}} \oplus Z(x, y, p) \tag{4.8}$$

Khi hoàn tác (‘undo‘), ta chỉ cần lặp lại đúng phép XOR đó để trả về mã băm ban đầu.

4.3.3 Cơ chế lưu trữ 3 loại Bound trong Transposition Table

Bảng Transposition Table lưu trữ các nút đã duyệt kèm độ sâu d và loại giá trị:

- **Exact (Chính xác):** Nút được duyệt trọn vẹn trong khoảng (α, β). Giá trị lưu trữ là điểm chính xác của nút.
- **LowerBound (Cận dưới):** Nhánh con gây ra hiện tượng β-cutoff ($v \geq \beta$). Giá trị thực tế của nút có thể còn lớn hơn điểm số đã lưu.
- **UpperBound (Cận trên):** Toàn bộ các nhánh con đều có điểm $< \alpha$ (All-node). Giá trị thực tế của nút không vượt quá điểm số đã lưu.

Chapter 5

Cơ Chế Cắt Mềm Không Gian Ứng Viên (Soft Cap)

5.1 Bán kính Lân cận Chebyshev $r \leq 2$

5.1.1 Khử 89% không gian rác ở khai cuộc

Bàn cờ 15×15 có 225 ô. Ở nước đi đầu tiên, nếu duyệt toàn bộ các ô trống, số lượng nhánh con ở gốc là 224 ô. Tuy nhiên, việc đặt một quân cờ ở góc $(0, 0)$ khi đối phương đang có quân ở $(7, 7)$ là hoàn toàn vô nghĩa về mặt chiến thuật.

Bộ sinh ứng viên ‘candidates.rs‘ áp dụng bán kính Chebyshev $r \leq 2$:

$$\mathcal{A}_{\text{cand}}(s) = \{(x, y) \in \text{Empty} \mid \exists (x_0, y_0) \in \text{Stones}, \max(|x - x_0|, |y - y_0|) \leq 2\} \tag{5.1}$$

Sau nước đi đầu tiên ở tâm $(7, 7)$, cửa sổ $r = 2$ tạo ra vùng vuông $5 \times 5 = 25$ ô, trừ ô tâm còn đúng 24 ứng viên. Tỉ lệ cắt giảm không gian ở gốc là:

$$\text{ReductionRate} = 1 - \frac{24}{224} \approx 89.29\% \tag{5.2}$$

5.2 Cơ chế Bảo Tồn Chiến Thuật Tuyệt Đối (Tactical Retention Rule)

5.2.1 Hiểm họa của giới hạn cứng (Hard Cap Loss)

Nếu áp dụng giới hạn cứng (Hard Cap = 15): thuật toán sắp xếp các ứng viên và chỉ lấy đúng 15 ô đầu tiên. Trong các thế trận phức tạp nơi đối thủ tạo ra nhiều mối đe dọa đồng thời, các nước chặn hiểm hóc có thể xếp ở vị trí 16, 17 và bị cắt bỏ thẳng tay. Điều này từng dẫn đến các ván thua tức tưởi trong các phiên bản thử nghiệm cũ.

5.2.2 Công thức Cắt Mềm Chiến Thuật (Soft Cap)

Để đảm bảo an toàn tuyệt đối, thuật toán định nghĩa tập chiến thuật sống còn:

$$\mathcal{T} = \mathcal{W}_{\text{own}} \cup \mathcal{W}_{\text{opp}} \tag{5.3}$$

Trong đó $\mathcal{W}_{\text{own}}$ là tập các nước thắng ngay của ta, và $\mathcal{W}_{\text{opp}}$ là tập các nước chặn đối thủ thắng ngay.

Định lý 5.1 (Quy tắc Bảo tồn Chiến thuật - Tactical Retention Invariant)**.** *Với giới hạn tham số cap c, tập ứng viên được chọn $\mathcal{C}^*$ tuân theo công thức:*

$$\mathcal{C}^* = \mathcal{T} \cup \mathit{Top}_{\max(0,c-|\mathcal{T}|)}\left(\mathcal{A}_{\mathit{cand}} \setminus \mathcal{T}\right) \tag{5.4}$$

Số lượng ứng viên thực tế được giữ lại là $\max(c, |\mathcal{T}|)$.

Nhờ cơ chế này, **độ bao phủ chiến thuật luôn đạt 100%** (‘TacticalRecall = 1.0‘): không bao giờ có một nước thắng ngay hay một nước chặn sống còn nào bị rơi rụng khỏi cây tìm kiếm.

Chapter 6

Hướng Dẫn Sử Dụng & Luồng Điều Phối Trong Engine Thực Tế

6.1 Kiến trúc Đa Luồng Web Worker & WebAssembly

Trong ứng dụng thực tế ‘CaroGame‘, việc tìm kiếm đệ quy sâu có thể tiêu tốn hàng trăm mili-giây. Nếu chạy trực tiếp trên luồng chính (Main Thread) của trình duyệt, giao diện người dùng sẽ bị đóng băng (UI Freeze), gây giật lag và mất khung hình.

Kiến trúc điều phối 3 tầng được thiết kế nhằm cô lập hoàn toàn tải tính toán:

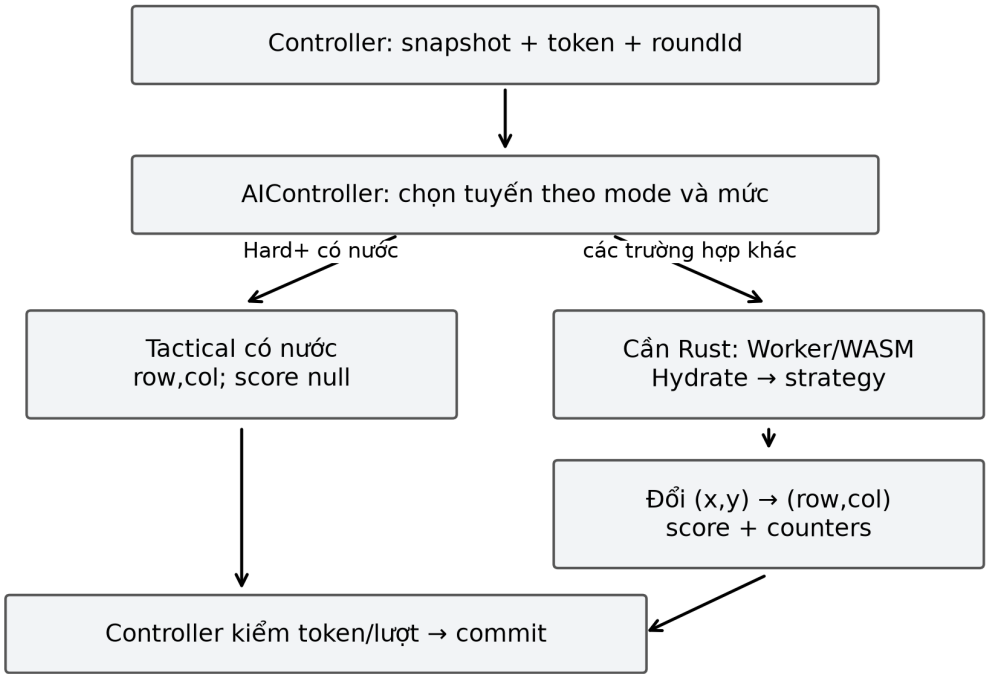


Figure 6.1: Sơ đồ luồng điều phối đa luồng giữa React UI, Web Worker và Rust WASM

1. **Tầng 1: Presentation (React + PixiJS):** Nhận thao tác nhấp chuột của người chơi, render bàn cờ

60 FPS, kích hoạt âm thanh Web Audio API.

2. **Tầng 2: Trung chuyển (TypeScript Web Worker):** Quản lý vòng đời yêu cầu AI, hủy bỏ các tìm kiếm quá hạn (‘cancelPending‘), xử lý các nước đi chiến thuật khẩn cấp (‘TacticalMoveResolver‘).

3. **Tầng 3: Engine Tính Toán Cốt Lõi (Rust WebAssembly):** Chạy mã máy nhị phân compiled từ ‘CaroGame/engine‘, thực thi Alpha-Beta, Zobrist Hashing và hàm lượng giá Heuristic tối ưu.

6.2 Cấu hình Chi Tiết 6 Cấp Độ Bot trong ‘CaroGame‘

Toàn bộ các tham số và thuật toán nghiên cứu được ánh xạ trực tiếp vào 6 cấp độ bot trong file ‘CaroGame/engine/src/ai.rs

Cấp độ Bot	ID	Chiến lược	Độ sâu d	Soft Cap	Cơ chế vận hành chi tiết
Easy	0	Random	0	None	Chọn ngẫu nhiên trong tập ô hợp lệ dựa trên PRNG SplitMix64 xác định.
Normal	1	Rule-based	1	None	Quét chiến thuật 1-ply: Ưu tiên thắng ngay $\succ$ Chặn ngay $\succ$ Lượng giá heuristic 1 nước.
Hard	2	Alpha-Beta	2	15	Alpha-Beta $d = 2$, áp dụng Available Span + Gapped Detection + Nhân Chebyshev.
VeryHard	5	Alpha-Beta	3	12	Alpha-Beta $d = 3$, Move Ordering nhanh, $\beta = 1.15$. Phản hồi < 40 ms.
Expert	3	Alpha-Beta	4	10	Alpha-Beta $d = 4$, Soft Cap bảo vệ chiến thuật tuyệt đối. Phản hồi < 90 ms.
Master	4	Alpha-Beta	5	8	Alpha-Beta $d = 5$, Transposition Table Zobrist 64-bit, cắt tỉa sâu. Phản hồi < 250 ms.

Table 6.1: Ma trận cấu hình 6 cấp độ bot tích hợp trong engine ứng dụng

Chapter 7

Tổng Hợp & Giới Hạn Nghiên Cứu

7.1 Bảng đối chiếu tổng hợp các cơ chế thuật toán

Thuật toán / Module	Cơ chế hoạt động chính	Hiệu quả thực nghiệm
Local Raycast	Quét tia 4 hướng quanh nước cuối tối đa 33 ô.	Đạt độ phức tạp $O(1)$, kiểm tra thắng $<$ 2 μs.
Available Span	Triệt tiêu về 0 nếu tổng không gian < 5.	Miễn nhiễm bẫy góc chết, giảm $>$ 15% node rác.
Gapped Scanning	Lookahead 1 ô trống nhận diện $X.XX$.	Tăng $+11.5\%$ tỉ lệ thắng trước bot ngây thơ.
Chebyshev Kernel	Nhân cự ly vuông đồng tâm $\max(\|x-7\|, \|y-7\|)$.	Đẳng hướng 4 trục, kiểm soát trung tâm từ khai cuộc.
Alpha-Beta Pruning	Cắt tỉa nhánh con khi $\alpha \geq \beta$.	Giảm $>$ 50–70% số nút duyệt so với Minimax.
Move Ordering	Sắp xếp nước tốt nhất lên đầu danh sách duyệt.	Giảm số node tiệm cận lý thuyết $O(\sqrt{b^d})$.
Zobrist Hashing	Băm XOR $O(1)$ nhận diện hoán vị trạng thái.	Khử trùng lặp nhánh, lưu trữ 3 loại TT Bounds.
Soft Tactical Cap	Giữ toàn bộ $W_p \cup W_o$ trước khi cắt mềm.	Tactical Recall đạt 100%, không mất nước thắng.

Table 7.1: Tổng hợp cơ chế và hiệu quả thực nghiệm của các thuật toán trong đồ án

7.2 Giới hạn nghiên cứu và Hướng phát triển

Mặc dù hệ thống đã đạt được hiệu năng và độ ổn định cao, nghiên cứu vẫn tồn tại một số giới hạn học thuật cần ghi nhận:

1. **Hiệu ứng đường chân trời (Horizon Effect):** Do Alpha-Beta dừng ở độ sâu cố định mà chưa tích hợp Quiescence Search sâu cho mọi chuỗi trao đổi quân, bot có thể không nhìn thấy đòn sát cục xảy ra ở độ sâu $d + 1$.

2. **Thuật toán tìm kiếm đòn sát cục liên tục (VCF/VCT):** Hướng phát triển tiếp theo là xây dựng module chuyên biệt cho Victory by Continuous Fours (VCF) và Victory by Continuous Threes (VCT) để giải quyết các thế trận cờ tàn đỉnh cao.

Bibliography

[1] C. E. Shannon, ”Programming a computer for playing chess,” *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.

[2] D. E. Knuth and R. W. Moore, ”An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[3] A. L. Zobrist, ”A new hashing method with application for game playing,” *Technical Report 88*, Computer Sciences Department, University of Wisconsin, Madison, 1970.

[4] R. E. Korf, ”Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[5] L. V. Allis, ”Searching for Solutions in Games and Artificial Intelligence,” Ph.D. dissertation, University of Limburg, Maastricht, 1994.